WOMEN’S SAFETY WEARABLE SOS BUTTON

Project Code & Explanation

Project goal: One press of an SOS button sends an emergency SMS containing GPS coordinates and a Google Maps link to saved emergency contacts.

Controller	ESP32 development board
GPS	NEO-6M GPS module
GSM	SIM800L 2G GSM module
Input / output	SOS push button and buzzer

Prepared as an implementation guide based on the uploaded project presentation. Replace the sample emergency numbers in the code before testing.

1. Project Overview

The ESP32 continuously reads NEO-6M GPS data. When the SOS button is held for about one second, the controller creates an emergency message with the latest valid latitude and longitude, then asks the SIM800L module to send the SMS to each saved contact. A short buzzer pattern indicates that the alert routine has run.

Main features

• One-button emergency alert.

• GPS coordinates converted into a Google Maps link.

• SMS sent to multiple saved phone numbers through the GSM network.

• No mobile internet connection is required; a working GSM network and SMS-capable SIM are required.

• A 60-second cooldown helps prevent repeated accidental alerts.

Important implementation notes

The uploaded presentation specifies an ESP32, NEO-6M, SIM800L, SOS button and buzzer, with GPS on UART2 (GPIO16/17) and GSM on UART1 (GPIO26/27). The code below follows those suggested connections. ■filecite■turn0file0■

Safety note: This is a student prototype, not a certified emergency device. Test it with consenting contacts and do not rely on it as the only way to seek help.

2. Components and Connections

Component	Pin / connection	Purpose / note
NEO-6M GPS	TX → GPIO16 (ESP32 RX2) VCC → suitable module supply GND → GND	Sends NMEA location data. Keep the antenna with a clear view of the sky.
SIM800L	TX → GPIO26 (ESP32 RX1) RX ← GPIO27 (ESP32 TX1, through level divider) GND → common GND	Sends SMS using AT commands. Use a dedicated 3.4–4.4 V supply capable of roughly 2 A peak current.
SOS button	One leg → GPIO4 Other leg → GND	Uses INPUT_PULLUP; pressed reads LOW.
Buzzer	Positive → GPIO5 Negative → GND	Short beeps provide local feedback. Use a transistor driver if the buzzer draws more current than the GPIO can safely supply.
Power	Common GND for modules	SIM800L power must not be taken from the ESP32 5 V or 3.3 V pin. Place a 1000 µF capacitor near the GSM module supply.

Supply voltage and logic-level requirements vary by breakout board. Check the exact module documentation before wiring.

3. Software Setup

Use Arduino IDE with ESP32 board support installed. Install the **TinyGPSPlus** library from the Arduino Library Manager. Select the correct ESP32 board and COM/USB port, then upload the sketch.

Before uploading

• Replace +91XXXXXXXXXX with each real emergency contact number in international format.

• Insert an active SIM card with SMS service and sufficient balance; verify that the SIM800L supports the local 2G network.

• Check the SIM800L power supply and common ground before powering the circuit.

• Open Serial Monitor at 115200 baud for diagnostic messages.

4. Complete Arduino Code

The sketch below uses a one-second button hold, reads GPS continuously, sends a Maps link, and waits 60 seconds before accepting another alert. If GPS has no valid fix, it sends a message saying that the location is unavailable rather than inventing coordinates.

```
#include <TinyGPSPlus.h>

TinyGPSPlus gps;

// ESP32 UART pins from the project wiring
HardwareSerial GPSSerial(2);   // UART2: RX=16, TX=17
HardwareSerial GSMSerial(1);   // UART1: RX=26, TX=27

const int SOS_BUTTON = 4;       // Button between GPIO4 and GND
const int BUZZER_PIN = 5;

const char* contacts[] = {
  "+91XXXXXXXXXX",              // Replace with emergency number 1
  "+91YYYYYYYYYY"               // Replace with emergency number 2
};
const int CONTACT_COUNT = sizeof(contacts) / sizeof(contacts[0]);

const unsigned long HOLD_TIME_MS = 1000;
const unsigned long COOLDOWN_MS = 60000;
unsigned long lastAlertTime = 0;
bool alertSentBefore = false;

unsigned long buttonDownAt = 0;
bool buttonHandled = false;

void beep(int count, int onMs = 120, int offMs = 120) {
  for (int i = 0; i < count; i++) {
    digitalWrite(BUZZER_PIN, HIGH);
    delay(onMs);
    digitalWrite(BUZZER_PIN, LOW);
    delay(offMs);
  }
}

void readGPS() {
  while (GPSSerial.available()) {
    gps.encode(GPSSerial.read());
  }
}

bool waitForGSM(const char* expected, unsigned long timeoutMs) {
  unsigned long start = millis();
  String response = "";

  while (millis() - start < timeoutMs) {
    readGPS();  // Keep GPS data flowing while waiting
    while (GSMSerial.available()) {
      char c = GSMSerial.read();
      response += c;
      if (response.indexOf(expected) >= 0) {
        return true;
      }
    }
    delay(5);
  }

  Serial.print("GSM response timeout. Received: ");
  Serial.println(response);
  return false;
```

```
}

bool sendSMS(const char* number, const String& message) {
  while (GSMSerial.available()) GSMSerial.read();

  GSMSerial.println("AT");
  if (!waitForGSM("OK", 2000)) return false;

  GSMSerial.println("AT+CMGF=1");   // SMS text mode
  if (!waitForGSM("OK", 2000)) return false;

  GSMSerial.print("AT+CMGS=\"");
  GSMSerial.print(number);
  GSMSerial.println("\"");

  if (!waitForGSM(">", 5000)) return false;

  GSMSerial.print(message);
  GSMSerial.write(26);              // Ctrl+Z sends the SMS

  bool sent = waitForGSM("+CMGS:", 15000);
  if (sent) {
    // Wait for final modem OK if available; not required for basic result
    waitForGSM("OK", 3000);
  }
  return sent;
}

String buildAlertMessage() {
  String msg = "EMERGENCY! I need help.";

  if (gps.location.isValid()) {
    double lat = gps.location.lat();
    double lon = gps.location.lng();

    msg += " My GPS location: https://maps.google.com/?q=";
    msg += String(lat, 6);
    msg += ",";
    msg += String(lon, 6);
  } else {
    msg += " GPS location is currently unavailable. Please contact me.";
  }

  return msg;
}

void triggerSOS() {
  Serial.println("SOS triggered");
  beep(2);

  String message = buildAlertMessage();
  Serial.println("Sending alert message:");
  Serial.println(message);

  int successCount = 0;
  for (int i = 0; i < CONTACT_COUNT; i++) {
    Serial.print("Sending SMS to ");
    Serial.println(contacts[i]);

    if (sendSMS(contacts[i], message)) {
      successCount++;
      Serial.println("SMS submitted to modem.");
    } else {
      Serial.println("SMS send failed; check SIM/network/power.");
    }
    delay(500);
  }

  if (successCount > 0) {
    beep(3);  // Confirmation that at least one send was accepted
  } else {
    beep(1);  // Failure/attention indication
  }

  lastAlertTime = millis();
  alertSentBefore = true;
}

void setup() {
  Serial.begin(115200);

  pinMode(SOS_BUTTON, INPUT_PULLUP);
  pinMode(BUZZER_PIN, OUTPUT);
  digitalWrite(BUZZER_PIN, LOW);

  GPSSerial.begin(9600, SERIAL_8N1, 16, 17);
  GSMSerial.begin(9600, SERIAL_8N1, 26, 27);

  delay(1000);
```

```
  Serial.println("Women's Safety SOS Wearable starting...");

  GSMSerial.println("AT");
  delay(500);
  while (GSMSerial.available()) {
    Serial.write(GSMSerial.read());
  }

  beep(1);
}

void loop() {
  readGPS();

  bool pressed = (digitalRead(SOS_BUTTON) == LOW);

  if (pressed && buttonDownAt == 0) {
    buttonDownAt = millis();
    buttonHandled = false;
  }

  if (!pressed) {
    buttonDownAt = 0;
    buttonHandled = false;
  }

  if (pressed && !buttonHandled &&
      millis() - buttonDownAt >= HOLD_TIME_MS) {
    buttonHandled = true;

    bool cooldownFinished =
      (!alertSentBefore || millis() - lastAlertTime >= COOLDOWN_MS);

    if (cooldownFinished) {
      triggerSOS();
    } else {
      Serial.println("Cooldown active; SOS not sent again yet.");
      beep(2, 60, 80);
    }
  }

  delay(5);
}
```

5. Code Explanation

Libraries and serial ports

TinyGPSPlus decodes GPS NMEA data. GPSSerial uses ESP32 UART2 on GPIO16/17 for the NEO-6M. GSMSerial uses UART1 on GPIO26/27 to communicate with the SIM800L.

Pin setup

The SOS button is configured as INPUT_PULLUP, so the input normally reads HIGH and becomes LOW when the button connects GPIO4 to GND. The buzzer is controlled by GPIO5.

GPS reading

readGPS() passes every incoming GPS character to TinyGPSPlus. The library updates latitude and longitude when it receives a valid fix. GPS may take time to acquire satellites, especially indoors.

Button hold and cooldown

The loop measures how long the button stays pressed. It triggers after one second, helping reduce accidental taps. After an alert attempt, a 60-second cooldown prevents another alert from being sent immediately.

SMS command sequence

sendSMS() sends AT to check modem communication, AT+CMGF=1 to select text mode, and AT+CMGS="number" to start a message. It waits for the > prompt, sends the message, then sends Ctrl+Z (ASCII 26).

Google Maps link

buildAlertMessage() formats a link using the current GPS latitude and longitude. The coordinates are inserted into https://maps.google.com/?q=LATITUDE,LONGITUDE.

Buzzer feedback

Two short beeps indicate that the SOS routine started. Three beeps indicate that the modem accepted at least one message for sending; this does not guarantee that the recipient received or read it.

6. Algorithm / Flow

• Start the ESP32 and initialise the button, buzzer, GPS UART and GSM UART.

• Continuously read GPS data and keep the latest valid position available.

• Check whether the SOS button is held for one second.

• If the cooldown has ended, create the emergency SMS.

• Send the SMS to every number in the contact list using SIM800L AT commands.

• Beep to provide local feedback and start the 60-second cooldown.

• Return to GPS reading and button monitoring.

7. Testing Procedure

Test	Expected result
Power-on test	One short beep; serial monitor shows the startup message.
GPS test outdoors	GPS eventually obtains a valid location; SMS contains coordinates and a Maps link.
Button test	Holding the button for about one second starts the SOS routine.
SMS test	Each configured contact receives the alert if GSM network, SIM and power are working.
Cooldown test	A second press within 60 seconds is ignored and a short warning beep is heard.

8. Troubleshooting

SIM800L repeatedly resets

The supply may not handle current peaks. Use a stable 3.4–4.4 V supply with roughly 2 A peak capability and a 1000 µF capacitor close to the module.

No SMS is sent

Check SIM activation, SMS balance, antenna, network coverage, baud rate, wiring, and Serial Monitor messages. Confirm that the local carrier still supports 2G service.

GPS location unavailable

Test outdoors with a clear sky view and allow enough time for a first fix. The sketch sends a fallback message if no valid GPS location exists.

ESP32 behaves unpredictably

Check common ground, avoid short circuits, and verify that the SIM800L RX input receives a suitable logic level through a divider. Never connect an incompatible voltage directly.

9. Limitations and Future Improvements

Limitations: GPS may not work well indoors; the SIM800L depends on available 2G service; a SIM and SMS service are required; and the modem’s message submission response is not proof of delivery. The project is not a certified safety product.

Future improvements: add a low-battery warning, battery monitoring, an enclosure designed as a pendant or wristband, a 4G-capable communication module where needed, a louder siren, and carefully tested location updates. Any live-tracking feature should protect the user's privacy and share location only with authorised contacts.

10. Conclusion

The Women’s Safety Wearable SOS Button demonstrates an embedded-system approach to emergency notification. The ESP32 monitors the SOS button, the NEO-6M provides coordinates, and the SIM800L sends an SMS alert to stored contacts. Proper power design, network testing and real-world safety testing are essential before any practical use.

Reference: The project structure and specified modules/connections are based on the uploaded presentation, “Women’s Safety Wearable SOS Button.” The code is an implementation example and should be tested on the actual hardware.